MERCHANT SERVICES · ANNUAL PRODUCT VISION · FY2027

The PSP that makes your acquiring strategy better

Even where we are not the acquirer. Where we are going, what we build to get there, how the product organisation has to work, and how we move a merchant estate off the engines without pretending the migration is free.

Document	PV-2027 · v1.2
Date	26 September 2026
Horizon	FY2027, with a three-year view
Audience	Product, Solutions, Technology, Operations
Implemented by	EA-001 v1.1 · EA-002 v1.0
Status	For leadership review

Contents

PART I — THE STRATEGY

01 The thesis

02 Why we must change

03 Why now, and why us

04 Pillar one — multi-acquiring, and the posture most PSPs cannot take

05 Pillar two — forward APIs, one abstraction over the ecosystem

06 Pillar three — accelerated funds management

07 Pillar four — payment instrument intelligence

08 The instrument intelligence graph

09 Pillar five — payment optimization, with proof

10 Credential portability, and a name doing two jobs

11 The economics of orchestrating volume we do not acquire

12 Four buyers, four different reasons to say yes

13 What winning looks like

14 The five strategic choices, and what they rule out

PART II — WHAT WE BUILD

15 The product model

16 Product setup — configure the business, not the machinery

17 Front-of-house products

18 Platform products, built as a service

19 Decision and experimentation, embedded in everything

20 AI-native decisioning, and why the order matters

21 The build sequence

PART III — THE OPERATING MODEL

22 The seven obligations

23 Processing, Optimization and the Decision Platform

24 The product team charter

25 Solutions and Product

26 Solution composition, in practice

27 Entitle by pattern, restrict by policy

28 Product metrics and the scorecard

29 Governance, investment, and the role of architecture

30 Business operations

31 Technology operations

32 Culture, and why it is downstream of ownership

33 The innovation model

PART IV — GETTING THERE

34 From engines to products

35 The six migration stages, and why shadow is the one that matters

36 Moving the merchant estate

37 The competitive narrative

38 What has to happen before what

39 FY2027 strategic outcomes

40 What we commit to in FY2027

41 Strategic risks

42 The compounding advantage

43 What success sounds like

PART

I — The strategy

01 The thesis
02 Why we must change
03 Why now, and why us
04 Pillar one — multi-acquiring, and the posture most PSPs cannot take
05 Pillar two — forward APIs, one abstraction over the ecosystem
06 Pillar three — accelerated funds management
07 Pillar four — payment instrument intelligence
08 The instrument intelligence graph
09 Pillar five — payment optimization, with proof
10 Credential portability, and a name doing two jobs
11 The economics of orchestrating volume we do not acquire
12 Four buyers, four different reasons to say yes
13 What winning looks like
14 The five strategic choices, and what they rule out

01

The thesis

> Every large merchant we want already uses more than one acquirer. Most of our competitors treat that as a problem to be closed. We should treat it as the opening.

> We are the payment operating system for enterprises that refuse to be single-acquirer — and we move their money faster than anyone else can, because we are the bank.

Five pillars carry that sentence. Two are consequences of what we already are and cannot be copied by a competitor who is not a bank. Two have to be earned, and will take the better part of the horizon to become defensible. The fifth is what makes the other four reachable at all.

FIVE PILLARS · TWO WE ARE BORN WITH, TWO WE EARN, ONE THAT ENABLES THE REST

STRUCTURAL · consequences of our banking structure, once productised

Acquirer neutrality

We win by orchestrating a multi-acquirer estate, not by being the only acquirer. A processor-economics competitor cannot adopt this without cannibalising its core.

Accelerated funds

Settlement, funding and working capital on our own rails, joined to payment visibility and underwriting. Competitors assemble this from intermediaries and capital arrangements.

EARNED · built, and only defensible once the evidence has accumulated

Payment instrument intelligence

What this instrument is, what it can do, where it can go, what it will cost — and how it actually behaves, learned from our own outcomes at our own volume.

Payment optimization

Every payment decision testable, measurable and reversible, with the gain proven against a reserved holdout rather than asserted.

FORWARD APIs · THE ENABLING PILLAR

one API abstraction over the ecosystem, and credentials that can move with the merchant

— The two structural pillars are why we can win. The two earned pillars are why we keep winning. The fifth is why any of it reaches a merchant without a project.

PILLAR	WHAT IT GIVES US	WHY A COMPETITOR STRUGGLES
Multi-acquiring	Revenue and visibility on volume we do not acquire, and the enterprise relationship at the orchestration layer rather than the processing layer	Their economics depend on being the processor. Neutrality cannibalises their core.
Forward APIs	One abstraction over the acquiring ecosystem, so adding an acquirer or a rail is our work rather than the merchant's	Their API exposes their processing model, and changing it is a breaking change for every integration.
Accelerated funds	Funding cycles measured in hours rather than days, and working capital underwritten against receivables we can already see	Non-bank competitors reach this through partner banks, capital arrangements or additional regulatory structure — possible, but not the same cost base.
Instrument intelligence	What an instrument is, what it can do, what it costs — and how it actually behaves	The first three layers are purchasable. The behaviour layer requires our volume and calendar time.
Payment optimization	Every decision testable, and the gain provable against a holdout	Many optimise. Very few can produce the number, the interval and the date.

THE OBJECTION TO MEET FIRST

"Routing volume to a competing acquirer earns us less than acquiring it ourselves. Is this strategy revenue-dilutive?"

Per transaction, yes. In total, no — provided we are honest about the counterfactual. These merchants are already multi-acquirer. Today we hold our share of their volume, see none of the rest, and have no position at the layer where the routing decision is made. Tomorrow we hold our share *plus* orchestration and optimization economics on the remainder *plus* the intelligence that comes from seeing all of it.

The alternative is not "we acquire it all". The alternative is a shrinking share and no visibility, while somebody else owns the orchestration layer and eventually the relationship.

02

Why we must change

> The organisation and the architecture both grew around engines. That model worked when a PSP's job was connecting a merchant to an acquirer. It is no longer the job.

What we run today is an authorization engine, a settlement engine, a risk engine, several legacy gateways, processor-specific integrations, separate operational systems and product-specific configuration — with capability duplicated across several of them and servicing fragmented across all of them. Each piece is individually defensible. Together they expose our internal structure to the merchant, and that is the actual problem.

FROM	TO	WHAT IT CHANGES FOR THE MERCHANT
Engine-centric	Product-centric	They buy an outcome, not access to a system
Single-acquirer thinking	Multi-acquiring	One integration reaches the acquiring ecosystem
Configuration-heavy onboarding	Capability-led onboarding	Six business answers, not forty-seven toggles
Static routing	Intelligent decisioning	Execution improves without them changing anything
Payment processing	Payment optimization	We are measured on their outcome, not our uptime
Settlement	Accelerated funds management	Money arrives faster and is easier to account for
Systems of record	Products with complete ownership	One owner answers for the whole experience
Siloed operations	End-to-end servicing	One question, one answer, one place
Roadmap delivery	Continuous experimentation	The platform gets better between releases

THE SENTENCE THAT SUMMARISES THE PROBLEM

A merchant today has to understand our engines, our processors, our connectors and, to a surprising degree, our organisational structure.

Every item in the left-hand column is a consequence of that. The transformation is not primarily a technology modernisation programme — it is a product transformation that technology modernisation enables. Conflating the two is how these programmes become re-platforming exercises that ship a new stack and the same experience.

03

Why now, and why us

WHAT HAS CHANGED	WHAT IT MEANS FOR US
Enterprises now treat single-acquirer as a concentration risk, not a simplification	The orchestration layer is being bought explicitly. If we do not sell it, an independent orchestrator will.
Optimization claims have become unfalsifiable and buyers have noticed	Proof is now a differentiator in its own right. We can build for proof; most cannot retrofit it.
Payment decisions are increasingly model-driven, and models need evidence nobody has been keeping	The counterfactual data we start logging this year is the only data that can train anything in two years.
Platforms and marketplaces are the fastest-growing segment, and funding speed is their weapon	Instant seller payout requires pre-funded float without a balance sheet behind it.
Settlement and funding expectations have compressed across every segment	A structural advantage we have historically under-sold.

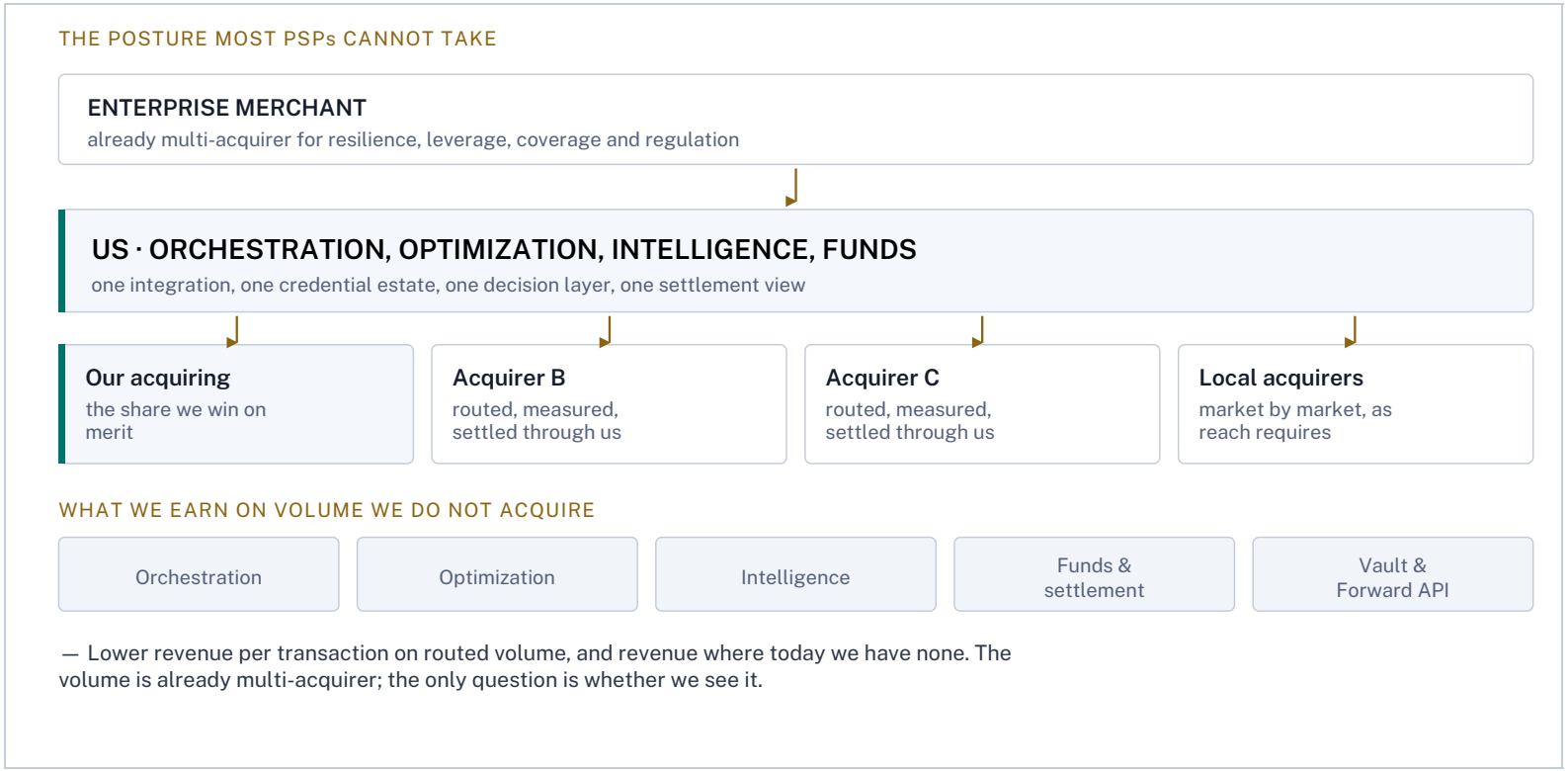
WHAT WE ARE NOT CLAIMING

We will not out-ship Stripe on developer surface area, and we should stop measuring ourselves that way. They have a decade of compounding on API design, documentation and ecosystem, and a product organisation built for it.

What we can be is the platform an enterprise treasurer, a head of payments and a CFO choose together — because it optimises across acquirers rather than within one, proves what it was worth, and moves the money faster. That is a different buying committee and a different competition.

Pillar one — multi-acquiring, and the posture most PSPs cannot take

> The strategic move is to stop competing for all of a merchant's volume and start competing for the layer that decides where it goes.



— Lower revenue per transaction on routed volume, and revenue where today we have none. The volume is already multi-acquirer; the only question is whether we see it.

What we actually sell to a multi-acquirer enterprise

CAPABILITY	THE PROBLEM IT REMOVES
One integration across every acquirer	Today each acquirer is its own integration, its own certification, its own release cycle.
One credential estate	Today the same cardholder is vaulted separately per acquirer, with no continuity and no shared lifecycle.
One decision layer	Today routing between acquirers is either static, or built in-house, or not done at all.
One settlement and reconciliation view	Today finance reconciles several acquirers separately and nets manually.
One evidence base	Today nobody can say which acquirer performs better for which instrument, because nothing is measured against a common baseline.
One commercial conversation	Today each acquirer negotiates separately and the merchant arbitrates.

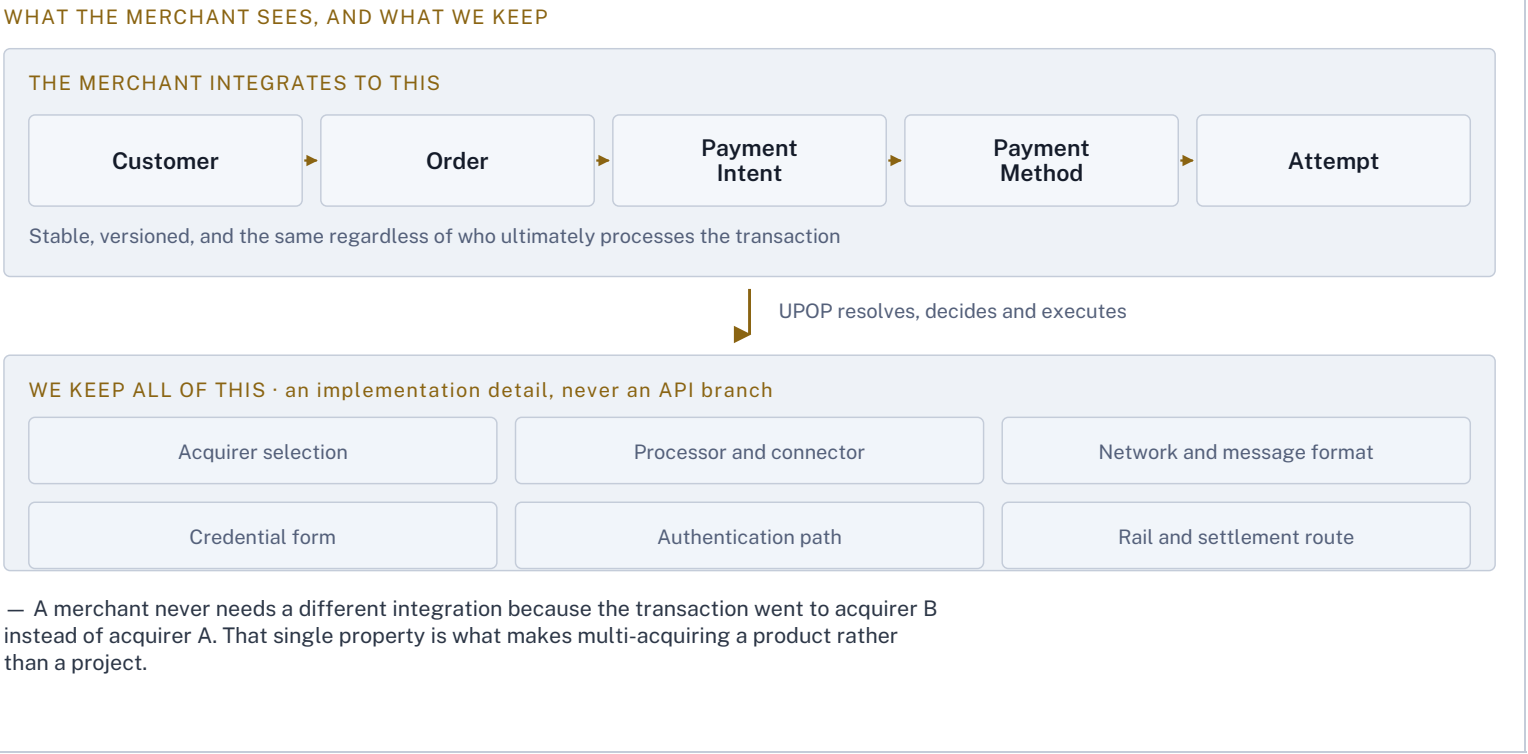
THE LAST ROW IS THE UNCOMFORTABLE ONE

If we orchestrate, we also become the place where our own acquiring is compared to a competitor's — measured, continuously, on the merchant's own traffic. That is exactly the discipline we are asking the organisation to accept, and we should expect the acquiring business to find it uncomfortable.

The answer is that the comparison happens anyway. The only question is whether it happens on our platform, with our measurement, and with us holding the relationship — or on someone else's.

Pillar two — forward APIs, one abstraction over the ecosystem

> The API model has to be designed around where payments are going, not around the processing model we happen to run. Acquiring should be an implementation detail.



— A merchant never needs a different integration because the transaction went to acquirer B instead of acquirer A. That single property is what makes multi-acquiring a product rather than a project.

PRINCIPLE	WHAT IT MEANS	WHAT IT RULES OUT
One payment abstraction	A merchant does not need a different integration because acquirer B handled the transaction instead of acquirer A	Processor-shaped endpoints, and per-acquirer certification for the merchant
Explicit lifecycle	Order, Payment Intent, Attempt, authorization, capture, refund, reversal and dispute are distinct resources	Overloading a single transaction object with a status enum and a dozen flags
Payment method above execution	The merchant chooses a method; we choose the rail	The rail appearing as an API branch, which makes every rail optimisation their decision
Attempt as a first-class resource	An attempt is one execution against one acquiring path	Retries, cascades, split tender and partial capture being modelled as special cases
Stable and versioned	The contract outlives the machinery behind it	A merchant re-integrating because we changed a processor

UPOP is the execution fabric that makes this work: it resolves the flow, invokes the decision points, executes the attempt and holds the transaction context. It should stay thin. The product owns the context and the modules stay independently deployable — an orchestration engine that accumulates business logic becomes the monolith the whole architecture was designed to avoid.

THE CAPABILITY THIS UNLOCKS

Multi-acquiring stops being a feature and becomes a property. Adding an acquirer, a rail or a local scheme is a connector and a capability entry — not an API change, not a merchant project, and not a certification the merchant has to run.

That is the difference between selling multi-acquiring and having it.

Pillar three — accelerated funds management

> The pillar we have historically under-sold, and the only one a competitor cannot buy their way into.

WHERE THE DAYS GO, AND WHICH ONES WE CAN REMOVE

		TYPICAL PSP	US
Authorization	T+0	●	●
Capture	T+0	●	●
Network clearing	T+1	●	●
Network settlement	T+2	●	●
Partner bank receipt	T+2	●	—
PSP reconciliation	T+2	●	●
Merchant funding	T+2 / T+3	●	●

AND WHAT THE BALANCE SHEET MAKES POSSIBLE ON TOP

Funding ahead of settlement
advance against captured, not-yet-settled receivables

Instant seller payout
for platforms, without a pre-funded float

Working capital
underwritten against a receivable we can already see

Multi-currency balances
held without forced conversion at settlement

— The advantage is the combination, not the charter: balance sheet, payment visibility, treasury and underwriting on one platform. Being a bank enables it; productising it is what makes it real.

The product surface

PRODUCT	WHO BUYS IT	WHY THEY CANNOT GET IT ELSEWHERE EASILY
Accelerated settlement	Every segment	Competitors route through a partner bank, which adds a hop we do not have
Funding ahead of settlement	Enterprises with working capital sensitivity	Requires balance sheet and the ability to underwrite the receivable
Instant seller payout	Platforms, marketplaces, PayFacs	Requires pre-funded float without a balance sheet; routine with one
Working capital against receivables	Growth merchants and platforms	Requires both the lending relationship and visibility of the receivable
Multi-currency balances	Cross-border enterprises	Requires holding balances rather than converting at settlement
Treasury integration	Enterprise finance functions	We already hold the relationship

One vision, three products

Settlement, funding and credit are not the same product with different timing. They carry different risk, different regulatory treatment and different P&L owners, and grouping them is how a payments team ends up originating lending without anyone deciding that it should.

ONE VISION, THREE PRODUCTS, THREE RISK PROFILES

Funds Management

Operational money movement

Settlement · reconciliation · balances · payout · FX · ledger

Operational risk

Payments

Accelerated Funds

A timing product

T+0 · same-day · instant · weekend and holiday funding

Liquidity risk

Payments + Treasury

Capital

A risk-bearing product

Receivables advance · working capital · credit · underwriting

Credit risk

Credit risk governance

— A receivables advance is credit, whatever the product page calls it. Grouped with settlement, a payments team can originate lending without anyone deciding that it should. The split is a risk and P&L boundary before it is a product one.

THE INTERNAL WORK THIS IMPLIES

Funding speed is a product, which means it has an owner, a price, an SLO and a reporting surface. Today it is closer to an operational characteristic that varies by arrangement.

Making it a product means: a stated funding time per segment and rail, a rate card for acceleration, metering of who used it, and a reconciliation view the merchant can see. That is the seven obligations applied to something the organisation currently treats as plumbing.

07

Pillar four — payment instrument intelligence

> The question: what is this instrument, what can it do, where can it go, what will it cost, and how should we use it? The first four are purchasable. The fifth is the asset.

LAYER	EXAMPLE FACTS	SOURCE	DEFENSIBLE?
Identity	BIN, range, issuer, product, country, network	Scheme and vendor data	No
Capability	Token eligibility, 3DS support, authentication methods, routing eligibility	Verified with processors and networks	Partly
Economics	Interchange qualification, scheme fees, processor cost, FX exposure	Published rules plus our own rates	Partly
Regulatory	Regulated debit status, cross-border flags, scheme classification	Published rules	No
Behaviour	**How this issuer actually responds, for this instrument, in this context — approval patterns, authentication posture, decline semantics, recovery response**	**Our own outcomes, at our own volume**	**Yes**

Multi-acquiring makes this pillar stronger, not weaker. An instrument observed across several acquirers tells us something no single-acquirer platform can see: how much of an outcome is the instrument, and how much is the acquirer. That separation is the difference between “this BIN declines” and “this BIN declines through that acquirer”, and only the second is actionable.

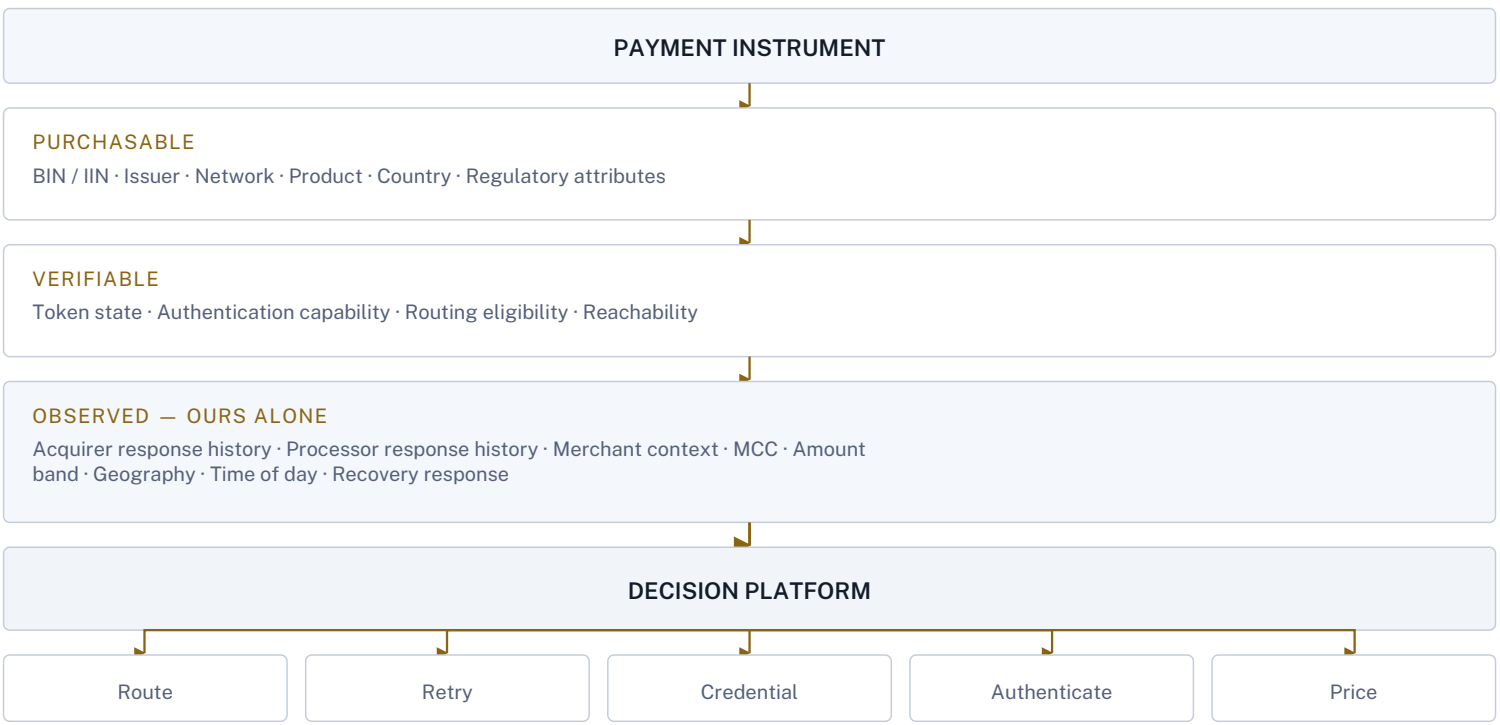
THE PRECONDITION

The behaviour layer is only real if decisions are logged with what else was permitted and what the fallback would have done. Without that, we have a record of what we happened to do — which can describe, but cannot optimise.

This is why Wave 0 of the transformation is evidence logging on the *existing* estate, before any migration. It is the only piece of this vision that cannot be started later.

The instrument intelligence graph

> The pillar stated as a data asset rather than a reference table. Three bands, and only the third is ours.



— The third band is the moat. It is a function of how many instruments we see, across how many acquirers, for how long — and multi-acquiring is what makes the acquirer dimension exist at all.

WHAT IT LETS US ANSWER	WHY IT NEEDS MULTI-ACQUIRING
Does this BIN decline, or does it decline through that acquirer?	The acquirer dimension does not exist in single-acquirer data
Is this a network token problem or an issuer problem?	Requires seeing the same instrument on both paths
Which acquirer performs best for this instrument class, in this market?	Requires several acquirers on comparable traffic
Is this decline recoverable, and on which path?	Requires observed recovery response across paths
What will this instrument actually cost on each route?	Requires cost observed rather than quoted

THE COMPOUNDING PROPERTY

Every pillar feeds this one. Multi-acquiring creates the acquirer dimension. Optimization creates the counterfactuals. The forward API means every merchant contributes on the same resource model, so the observations are comparable. Accelerated funds adds the cost-of-money dimension to the economics layer.

That is why the moat is the combination rather than any single pillar — each one on its own is reachable by a competitor with funding.

Pillar five — payment optimization, with proof

> Optimization is a crowded claim. Proof is not.

CLAIM	HOW CROWDED	OUR POSITION
“We have smart routing”	Everyone	Not a differentiator. Stop leading with it.
“We use machine learning”	Everyone	Unfalsifiable and increasingly heard as noise.
“We optimise across acquirers, not within one”	Few	Structural. Follows from pillar one.
“We can change a payment decision in days”	Some	True, meaningful, and reachable by others who build a control plane.
“Here is what we were worth last quarter, measured against a holdout”	**Almost nobody**	**The sentence to build the story on.**

The machinery behind that last row already exists in design: UPOP as the orchestration and execution layer, the decision platform as the place every strategy is registered and executed, POEM for decision-point experimentation with orthogonal assignment, and the evidence contract that makes off-policy evaluation possible. What has not existed is the discipline to reserve a holdout and report against it.

THE ONE DECISION THIS PILLAR NEEDS THIS QUARTER

A permanent, statistically valid control population — reserved before the first experiment runs, excluded from experiments and from exploration, with a creation date nobody can edit.

The ***percentage*** is an implementation parameter, derived from TPV, minimum detectable effect, segmentation and experiment volume. Two percent is a working assumption; for some populations 0.5% is ample and for fragmented ones it is not enough. The ***immutability*** is not a parameter. A holdout whose size is renegotiable is a holdout that gets negotiated down, one reasonable quarter at a time, until it no longer measures anything.

It cannot be created retroactively, and without it we can measure individual experiments but never the platform. Every other decision in this document can wait a quarter. This one degrades every week it waits.

Credential portability, and a name doing two jobs

> Worth resolving now, because it is already causing confusion in review: "Forward API" currently refers to two different things.

THE TWO MEANINGS	WHAT IT ACTUALLY IS	WHERE IT LIVES
Forward API — credential forwarding	The capability to present a credential vaulted with us through another acquirer, processor or PSP, on instruction	A capability inside Credential Management
Forward APIs — the forward-looking model	The resource abstraction above — Customer, Order, Payment Intent, Method, Attempt — that makes acquiring an implementation detail	The Payment Processing API surface

Recommendation. Keep ***Forward API*** for credential forwarding, where it already has currency with the vault team and with enterprise prospects. Call the other thing the ***Payment API*** or the ***payment resource model***. Two capabilities with one name will cost more in misunderstanding than either saves in branding.

Four API layers, cleanly named

LAYER	CONTRACT	AUDIENCE
Payment API	The merchant-facing resource model — Customer, Order, Payment Intent, Payment Method, Attempt	Merchants and their developers
Forward API	Credential portability across acquiring ecosystems	Merchants, and PSPs or banks consuming our vault
Decision API	The platform contract for registering and executing a decision	Internal product teams
Connector API	The acquirer, processor and rail integration contract	Partner and connector engineering

Four names, four audiences, four versioning lifecycles. Most API confusion in a platform this size comes from one surface trying to serve two of these at once.

Why credential portability matters strategically

CREDENTIALS THAT ARE NOT HOSTAGE TO US

Merchant vaults with us
one credential estate, one tokenisation, one lifecycle

Forward API presents it
to our acquiring, to another acquirer, to another PSP, on instruction

Credential continuity
the same cardholder resolves the same way regardless of who acquires

Lifecycle stays central
updater, network token status, consent and mandate in one place

Exit is possible
the merchant can leave with their credentials — and mostly does not

— Every other PSP makes the vault the lock-in. Making it portable is a posture only a platform confident in its optimization can take — and it is the single most credible signal that we mean the multi-acquiring story.

WITHOUT CREDENTIAL FORWARDING	WITH IT
A credential vaulted with us is usable only through us	The same credential can be presented through any acquirer, on instruction
Multi-acquiring means multiple vaults and multiple tokenisations	One vault, one lifecycle, one identity resolution
A cardholder looks like a different customer at each acquirer	One cardholder, one identity, across the estate
Account updater and token lifecycle are duplicated and diverge	Run once, centrally, reflected everywhere
Leaving us means re-collecting every credential	Leaving us is possible — which is why they mostly will not

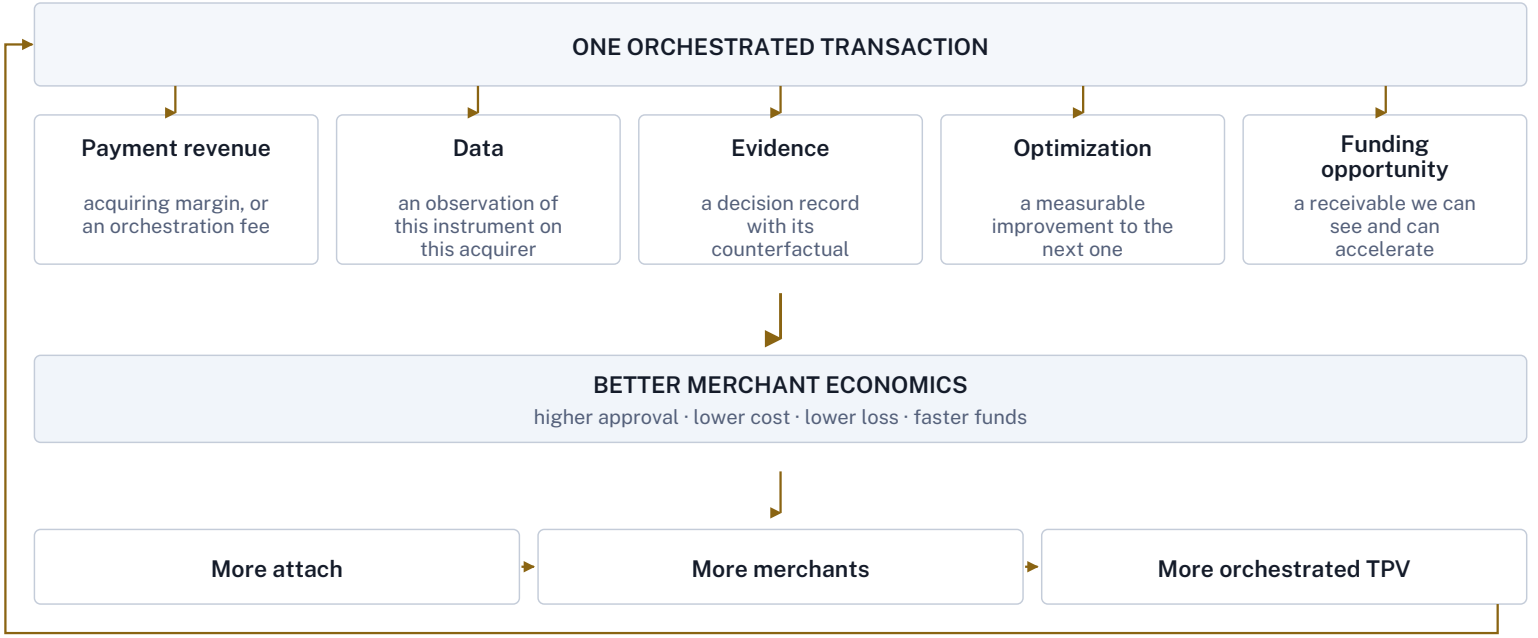
THE POSTURE, AND WHY WE CAN AFFORD IT

Every other PSP makes the vault the lock-in, because it is the cheapest lock-in available. It works until the merchant resents it, and enterprise procurement now prices that resentment in.

We can take the opposite position precisely because retention rests on optimisation we can prove and funding speed a competitor cannot match. It also opens the fourth buyer segment: a PSP or bank will never consume our vault while doing so means surrendering their credential estate.

The economics of orchestrating volume we do not acquire

> The document has so far asserted that this is accretive in total even though it is dilutive per transaction. That claim needs an equation, not a sentence.



— Five returns per transaction, only one of which is payment revenue. A model priced solely on the first will look thin; a model priced on all five is the actual proposition.

The equation

$$\text{Today} \quad R_0 = T \cdot a_0 \cdot m$$

$$\text{Future} \quad R_1 = T \cdot [\, a_1 \cdot m + (o - a_1) \cdot f + o \cdot k \,]$$

T	enterprise TPV in scope	a_0	our acquiring share today
a_1	our acquiring share after	m	acquiring net margin, bps
o	share of T we orchestrate	f	orchestration fee on non-acquired volume, bps
c	cannibalisation of a_0, so $a_1 = a_0(1-c)$	k	attach per orchestrated dollar — optimization, intelligence, funds — bps

$$\text{Break-even orchestrated share} \quad o^* = [\, a_0 \cdot c \cdot m + a_0 (1-c) \cdot f \,] \;/\; (f + k)$$

Where the crossover sits

Required orchestrated share of enterprise TPV to match today's revenue, at an illustrative 40% acquiring share, 35 bps acquiring net margin and an 8 bps orchestration fee. Because we orchestrate at least what we already acquire, any cell at or below 40% is cleared on day one.

CANNIBALISATION	ATTACH 3 bps	ATTACH 6 bps	ATTACH 9 bps	ATTACH 12 bps
10% of our acquired share moves	38.9%	**30.6%**	**25.2%**	**21.4%**
20% moves	48.7%	**38.3%**	**31.5%**	**26.8%**
30% moves	58.5%	46.0%	**37.9%**	**32.2%**
40% moves	68.4%	53.7%	44.2%	**37.6%**

THE FINDING, AND IT IS NOT THE OBVIOUS ONE

Attach rate dominates the orchestration fee. Moving attach from 3 to 12 bps cuts the required orchestrated share by roughly thirty points at every cannibalisation level. Moving the orchestration fee does comparatively little.

The consequence is direct: *if we price orchestration as a toll, the model is marginal. If we price the attached products — optimization, intelligence, accelerated funds — it clears easily even at heavy cannibalisation.* That is the same conclusion the entitlement section reached from the other direction, and the two should be decided together.

WHAT THIS MODEL DELIBERATELY LEAVES OUT

Incremental infrastructure and connector cost, migration cost, dual-run cost, funding cost of acceleration, and any commercial incentive used to win the orchestration mandate. Each is a real line and none is in the equation above.

The three inputs are illustrative placeholders, not our numbers. Finance owns a_0, m and f; Product owns k, and k is the one this strategy can actually move. The companion model lets all four be changed. The companion workbook `psp-multi-acquiring-economics.xlsx` is live — change any input and the crossover and the sensitivity grid recompute.

One framing correction worth making explicitly: the counterfactual is not “we acquire everything”. That was never available. It is “we hold today’s share and see nothing else”.

Four buyers, four different reasons to say yes

SEGMENT	WHAT THEY ARE ACTUALLY BUYING	LEAD PILLAR	WHAT WOULD LOSE IT
Direct enterprise merchants	Freedom from single-acquirer concentration, plus proof that optimisation is worth what we say	Multi-acquiring	Lock-in behaviour, or an optimisation claim we cannot evidence
Platforms, PayFacs and ISVs	Speed of seller boarding and speed of seller money	Accelerated funds	Boarding friction, or payout timing we cannot commit to
Other PSPs and banks	Infrastructure they do not want to build — intelligence, decisioning, vault and Forward API	Instrument intelligence	Any hint that consuming us means surrendering their estate
Internal JPM lines	A payments platform they can build on without becoming a payments team	Payment optimization	Treating internal consumers as stakeholders rather than customers

Inside an enterprise, seven people have to say yes

WHO	WHAT THEY CARE ABOUT	WHICH PILLAR SPEAKS TO THEM
Head of Payments	Acceptance rate, optimisation, acquirer leverage	Multi-acquiring and optimization
CTO	Integration effort, reliability, migration cost	Forward APIs
CFO or Treasurer	Funding timing, FX, settlement predictability, working capital	Accelerated funds
Procurement	Commercial flexibility, exit terms, concentration risk	Multi-acquiring and credential portability
Risk and Compliance	Fraud, disputes, regulatory posture, auditability	Decision evidence and policy
Developer	API quality, docs, sandbox, time to first transaction	Forward APIs and developer experience
Operations	Visibility, servicing, reconciliation	Decision trace and unified servicing

The strategy has something for each of them, which is unusual and worth exploiting: most competitors are strongest with one or two of these seven. The commercial narrative should be assembled per buyer rather than delivered as one deck.

THE THIRD SEGMENT IS THE STRATEGIC ONE

Selling intelligence, decisioning and vault to other PSPs and banks is a genuinely different motion — infrastructure as a product rather than acquiring as a service. It is lower revenue per unit and much higher leverage, and it makes the behaviour layer in pillar three compound faster because it widens the observation base beyond our own acquiring.

It is also the segment that is impossible without Forward API and without acquirer neutrality being genuine rather than positioning. A competitor asked to consume our vault will test that in the first meeting.

What winning looks like

> Measures, not targets. The targets belong to the planning cycle; the measures belong to the strategy, and choosing the wrong ones is how a good strategy gets managed into a bad one.

DIMENSION	THE MEASURE	THE ANTI-MEASURE WE SHOULD REFUSE
Multi-acquiring	Share of orchestrated volume we do not acquire	Share of volume we acquire directly — it punishes the strategy
Optimization	Measured lift against the reserved holdout, per quarter	Approval rate in isolation — it hides the cost side
Intelligence	Months of propensity-logged evaluable history, and decisions covered	Size of the BIN database — anyone can buy rows
Funds	Time from capture to merchant availability, by segment	Settlement volume — it does not measure speed
Product model	Proportion of products owning all seven obligations	Feature count or release count
Platform	Products consuming shared services versus forking them	Platform team utilisation
Migration	Volume on the new platform, and cohorts fully cut over	Merchants “enabled” — which counts configuration, not traffic
Commercial	Attach rate across product lines per client	Number of products in the catalogue

THE MEASURE THAT WILL CAUSE THE MOST ARGUMENT

Share of orchestrated volume we do not acquire. It is, on its face, a measure of volume going to competitors, and it will be read that way by anyone compensated on acquiring share.

It is the right measure precisely because of that. If the organisation cannot hold this number as a success metric, it has not accepted pillar one, and the multi-acquiring strategy will be quietly undone by incentives regardless of what this document says.

PART

II — What we build

15 The product model

16 Product setup — configure the business, not the machinery

17 Front-of-house products

18 Platform products, built as a service

19 Decision and experimentation, embedded in everything

20 AI-native decisioning, and why the order matters

21 The build sequence

The five strategic choices, and what they rule out

> A strategy that forecloses nothing is a list of aspirations. These are the five choices this document is actually making, each with the thing it gives up.

#	THE CHOICE	WHAT WE GIVE UP BY MAKING IT
1	**We optimise across acquirers rather than maximise our own acquiring share.**	Per-transaction revenue on volume we route away, and the simplicity of an acquiring-share target
2	**We expose a stable payment abstraction and make acquiring an implementation detail.**	The ability to surface processor-specific capability directly, and some short-term integration shortcuts
3	**We monetise faster funds movement rather than treat it as infrastructure.**	Funding speed as a free differentiator in deals, and the comfort of an operational SLA instead of a priced product
4	**We build proprietary instrument behaviour intelligence from our own outcomes.**	The speed of simply buying reference data and calling it intelligence
5	**We make no optimisation claim without experimental evidence.**	Marketing velocity, and the ability to ship on intuition when a deal is waiting

What we will not do

WE WILL NOT	BECAUSE
Build another processor-shaped API	It makes every acquirer addition a merchant project
Optimise only our own acquiring book	It is the strategy we are transforming out of
Build per-product decision engines	Two definitions of a control group means no shared evidence
Let Solutions commission one-off capabilities	The roadmap ends up written by whichever deal was loudest
Treat merchant migration as a technology project	No merchant re-integrates for our architecture
Treat faster funding as an operational SLA only	An unpriced advantage is a cost centre
Measure product success primarily through feature delivery	It is the metric that makes all of the above rational

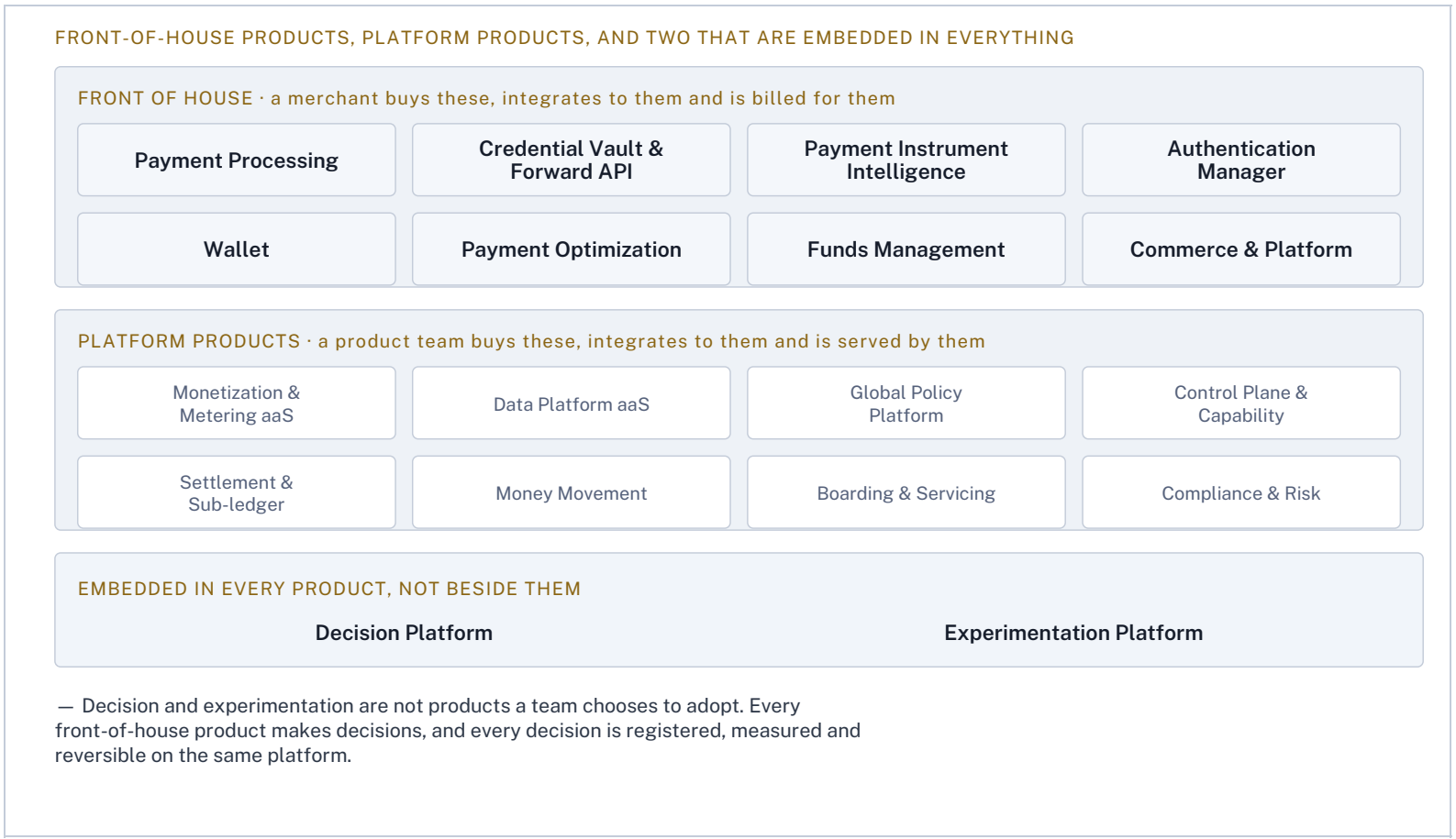
THE CHOICE THAT WILL BE TESTED FIRST

Choice one. It will be tested the first time routing moves meaningful volume from our own acquiring to a competitor's, and somebody compensated on acquiring share escalates it.

If that escalation succeeds, the strategy is over and the rest of this document is decoration. The economics in the next section exist so that the argument can be had on numbers rather than on territory.

The product model

Two kinds of product, and two platforms that are not optional for anyone.



— Decision and experimentation are not products a team chooses to adopt. Every front-of-house product makes decisions, and every decision is registered, measured and reversible on the same platform.

	FRONT-OF-HOUSE PRODUCT	PLATFORM PRODUCT
Customer	A merchant, platform or partner	Another product team
Bought how	Entitlement under a commercial agreement	Adoption under an internal service contract
Integrated by	The client's developers	The consuming product's engineers
Success measured as	Client outcome and revenue	Consumer adoption, and the absence of forks
Failure mode	A capability sold that does not work in that market	A product team building its own copy

THE DISTINCTION THAT IS USUALLY LOST

A platform product is not "infrastructure" and its team is not a shared service function. It has customers, a roadmap those customers can plan against, a support path with a real response time, and a usage model. Monetization and metering built the way Metronome builds it; a data platform that product teams push to and pull from for their own model, not a central warehouse they file requests against.

The test is simple and unforgiving: can a consuming product team plan a quarter against the platform product's roadmap? If not, it is not a product, and it will be forked.

Product setup — configure the business, not the machinery

> Today a merchant configures processor, routing, retry, fraud, 3DS, tokenisation, settlement, reporting, billing and credentials. In the target model they answer six questions about their business and we derive the rest.

CONFIGURE THE BUSINESS, NOT THE MACHINERY

Platform defaults
what any client of this shape gets, set once by Product

Solution defaults
what a client in this industry gets, set by Solutions

Merchant policy
what this client has chosen to restrict or require

Transaction context
what is true of this specific payment

Decision
the strategy that runs, within everything above

WHAT THE MERCHANT ACTUALLY PROVIDES AT BOARDING

Business and legal entity	Countries and currencies	Payment methods wanted
Risk profile	Settlement preferences	Commercial terms

— Six business answers, not forty-seven capability toggles. Everything underneath is derived, and an advanced client can override where policy permits.

LAYER	SET BY	CHANGES AT THE PACE OF
Platform defaults	Product	Rarely — these are the platform's opinions
Solution defaults	Solutions	Per industry or segment, as the market moves
Merchant policy	The client, within what is permitted	As their business changes
Transaction context	The transaction itself	Every payment
Decision	The bound strategy, inside all of the above	Continuously, through experimentation

THE COMMERCIAL CONSEQUENCE NOBODY HAS PRICED

If capabilities are broadly entitled by default and narrowed by policy, then **entitlement stops being the commercial lever**. You cannot sell a premium tier defined by which capabilities a client can reach, because they can reach almost all of them.

The commercial model therefore has to be usage-based and outcome-based — metered consumption, and share of measured gain — rather than tier-based. That is a direct consequence of the setup philosophy and it needs to be decided alongside it, not discovered afterwards when someone asks what the Enterprise tier contains.

Front-of-house products

PRODUCT	WHAT IT OWNS END TO END	WHAT IT SELLS
Payment Processing	Order, intent, attempt; orchestration through UPOP; rail and acquirer integration; capture, refund and void; its own setup, metering, pricing, reporting and servicing	Acceptance across every method and rail, through one integration
Credential Vault & Forward API	Vault, surrogate and state; network tokenisation; account updater; consent and mandate; identity resolution; credential portability	One credential estate that works across acquirers and can leave
Payment Instrument Intelligence	Identity, capability, economics, regulatory and behaviour layers; the reference and observed view of every instrument	Knowing what an instrument will do before you send it
Authentication Manager	3DS and SCA orchestration; exemption strategy; step-up placement; delegated authentication; the authentication outcome record	Approval and conversion, traded deliberately
Wallet	Stored value, balances, consumer-side credential storage and the wallet lifecycle	Balance-based acceptance and closed-loop economics
Payment Optimization	The strategies bound at every decision point, the objective function, and the measured readout against holdout	Continuous improvement, with the number and the interval
Funds Management	Settlement timing, acceleration, balances, payout, working capital and the reconciliation view	Money that arrives faster and is easier to account for
Commerce & Platform	Subscriptions and invoices; marketplace, PayFac and connected accounts; split, reserve and seller payout	Revenue recovery, and payments for other people's sellers

WHAT "END TO END" MEANS HERE, CONCRETELY

Payment Processing does not hand billing to a billing team, reporting to a reporting team, setup to an implementation team and support to a service team. It owns the client's experience of Payment Processing from the moment it is configured to the moment it is invoiced and supported — using platform products to do it, but owning the outcome.

This is the single largest change from how the organisation works today, and section 15 is the mechanism.

Platform products, built as a service

PLATFORM PRODUCT	ITS CUSTOMERS GET	THE MODEL TO BUILD TOWARD
Monetization & Metering aaS	A metering endpoint, a rate card model, rating, and per-product usage and revenue visibility	Metronome — usage-based billing as an API, not a batch process
Data Platform aaS	Push their own events, pull their own model, own their own semantics, with lineage and retention handled	A platform products publish to and query, not a central warehouse they queue behind
Global Policy Platform	Express and evaluate what is permitted, with the authority ladder resolving conflicts	Policy as a service with a bitemporal record of what was in force when
Control Plane & Capability	Register what they can do, and publish it globally as one signed epoch	The four-plane model — capability, entitlement, preference, policy
Settlement & Sub-ledger	Positions calculated and recorded, reconcilable against their own product view	Three contracts, not one system
Money Movement	A single place transfers are instructed and executed	No product initiates a transfer itself
Boarding & Servicing	Their own setup and servicing steps sequenced into a client journey they do not have to build	An orchestrator that sequences contracts, never one that contains product logic
Compliance & Risk	KYC, KYB, screening, underwriting and exposure as a service	Gates a product calls, not a team a product waits for

THE FAILURE THAT ENDS THIS MODEL

A platform product that cannot move at the pace its consumers need will be forked, and the fork will be called a thin wrapper. Two years later it is a second implementation with different semantics.

The defence is not a rule forbidding forks. It is a roadmap consumers can plan against, plus a visible, time-boxed exception when the platform genuinely cannot deliver in time. A forbidden fork with no exception path happens anyway and is simply not called one.

Decision and experimentation, embedded in everything

> These two are not products a team chooses to adopt. Every front-of-house product makes decisions, and every decision is registered, executed, measured and reversible on the same platform.

PRODUCT	ITS DECISION POINTS	WHAT IT REGISTERS AS STRATEGIES
Payment Processing	Routing, retry, capture timing, rail selection	Processor and acquirer selection, retry placement and cascade
Credential Vault	Credential selection, refresh timing	Which credential form to present, when to refresh
Authentication Manager	Authenticate or not, exemption choice, step-up placement	Exemption strategy, challenge thresholds
Instrument Intelligence	Enrichment and eligibility resolution	Which capability facts to trust, and for how long
Funds Management	Acceleration eligibility, payout timing	Funding strategy by segment and exposure
Commerce & Platform	Retry schedule, recovery path, split and reserve logic	Revenue recovery strategies, reserve policy

The consequence is that **no product builds a rules engine**. A product author writes a strategy; the decision platform executes it, assigns it, records the evidence and makes it experimentable. This is the rule that produces a shared evidence base rather than six private ones, and it is the hardest one to hold when a product team already has a working engine.

THE MIGRATION NOBODY WANTS

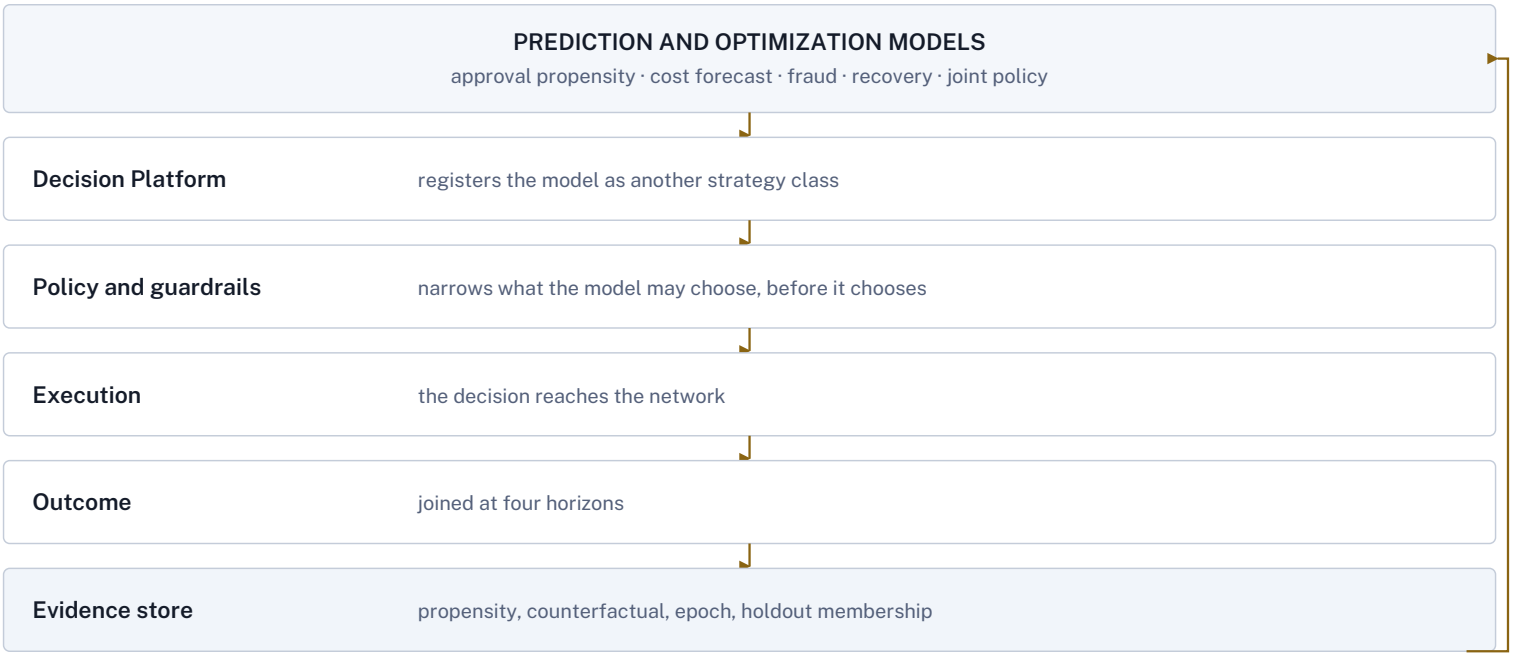
Fraud and routing both have mature decision engines today. Moving them is the hardest work in the programme, and the honest position is that the ***models*** stay with their owning product while the ***strategies*** — the mapping from score or signal to action — move to the shared platform.

If they do not move, there are two definitions of a control group and no shared evidence, which removes most of the reason to build any of this.

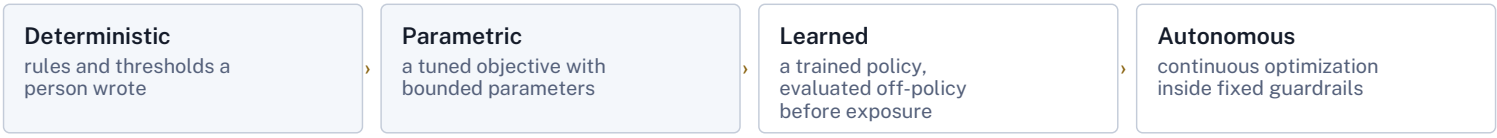
AI-native decisioning, and why the order matters

> This document has described almost everything needed to build an AI-driven PSP without using the word. That is deliberate up to a point, and past that point it is an omission.

AI SITS ABOVE THE EVIDENCE INFRASTRUCTURE, NOT INSTEAD OF IT



THE AUTONOMY LADDER, AND WHERE THE BANK CONSTRAINT BITES



— Every rung above deterministic is a model under SR 11-7 governance: documented, independently validated, monitored. That is slower than a competitor without a model risk function — and it is the reason the evidence layer matters more for us, not less.

THE COMMON APPROACH	OURS
Buy or build a model, then find data to feed it	Build the evidence layer, then the model has something to stand on
AI as a product claim	AI as a strategy class registered at a decision point, like any other
Model performance measured offline	Measured against a reserved holdout, in production
A private path to production for models	The same registration, gates and guardrails as a deterministic strategy
“We use AI”	“This model was worth N bps, interval, date, and here is the decision record”

THE CONSTRAINT THAT IS OURS AND NOT A COMPETITOR’S

Every rung above deterministic is a model under SR 11-7 governance: documented, independently validated, monitored, with a named owner. That is genuinely slower than a competitor with no model risk function, and pretending otherwise would be the fastest way to lose the argument with Risk.

It is also why the evidence layer matters more for us rather than less. A model that arrives with propensity-logged history, a counterfactual, an off-policy evaluation and a holdout readout is a model validation can actually approve. One that arrives with an offline accuracy score is a model that waits two quarters.

The build sequence

WHEN	WHAT	WHY IN THIS ORDER
Now, before anything else	Evidence logging on the current estate; the reserved holdout; the economic objective expressed as an exchange rate	None can be created retroactively, and none requires migration
First half	Control plane and capability catalog; policy platform; entitlement patterns; the new edge	Everything downstream resolves against these
First half	Decision platform execution and registration; UPOP orchestration on the new edge	The mechanism the optimization pillar depends on
Second half	Forward API; credential state and surrogate split; multi-acquirer routing across two live acquirers	Pillar one becomes demonstrable rather than described
Second half	Experimentation platform and the first measured readout; funds acceleration as a priced product	The proof and the pricing arrive together
Following year	Instrument intelligence behaviour layer; learned strategies; marketplace and platform payments; infrastructure segment	Each depends on evidence that only exists once the earlier waves have run

THE SEQUENCING PRINCIPLE

Anything that cannot be created retroactively goes first, regardless of how unglamorous it is. Anything that can be bought or built later goes later, regardless of how much it is being asked for.

Applied honestly, that principle puts propensity logging and a reserved holdout ahead of every feature on the roadmap — which is not a comfortable conversation, and is the right one.

PART

III — The operating model

22 The seven obligations
23 Processing, Optimization and the Decision Platform
24 The product team charter
25 Solutions and Product
26 Solution composition, in practice
27 Entitle by pattern, restrict by policy
28 Product metrics and the scorecard
29 Governance, investment, and the role of architecture
30 Business operations
31 Technology operations
32 Culture, and why it is downstream of ownership
33 The innovation model

The seven obligations

> The anti-silo mechanism. A product that owns fewer than seven is a component with a product manager attached, and the gaps are where client experience goes to die.

THE SEVEN OBLIGATIONS · a product owning fewer is a component with a product manager attached

1	**Setup**	how a client is configured for it, and how fast
2	**Integration**	the API, the SDK, the docs, the sandbox
3	**Execution**	the runtime behaviour and its SLO
4	**Monetization**	pricing, packaging, rate cards, the commercial model
5	**Metering**	emitting billable events accurately, every time
6	**Reporting**	what the client sees, and what we see
7	**Servicing**	support, change and incident ownership after launch

AND THE SAME SEVEN, TRANSLATED FOR A PLATFORM PRODUCT

Setup
how a product team onboards to the service

Integration
the service contract, its SDK and its documentation

Execution
the SLO the consuming product can plan against

Monetization
internal chargeback, or an explicit decision not to charge

Metering
usage visible per consuming product

Reporting
what consuming teams can see about their own usage and health

Servicing
a real support path, with a real response time

— A platform product's customer is another product team, and it owes them the same seven things a merchant is owed. That is what makes "as a service" real rather than a label.

Why silos form, and what actually dissolves them

THE SILO	HOW IT FORMS	WHAT THE OBLIGATIONS CHANGE
Build versus run	A product owns execution but not servicing	Servicing is obligation seven. The team that built it answers for it.
Product versus billing	A product owns the capability but not its metering or price	Monetization and metering are obligations four and five.
Product versus implementation	A product owns the API but not how a client is set up	Setup is obligation one. Time-to-live is a product metric.
Product versus reporting	A product owns the behaviour but not what the client sees of it	Reporting is obligation six.
Platform versus product	Platform owns a service but not its consumers' success	All seven apply to platform products too, with product teams as customers.

CULTURE DOES NOT FIX THIS. OWNERSHIP DOES.

Every organisation that has had a silo problem has tried to solve it with a values statement first. It does not work, because silos are not an attitude — they are what happens when the thing a person is accountable for is smaller than the thing the client experiences.

The seven obligations are the structural fix. Culture, in section 20, is what becomes possible once the structure is right — not a substitute for getting it right.

Processing, Optimization and the Decision Platform

> Three names have a claim on the same transaction. Left implicit, this becomes the defining ownership argument of the next two years.

WHO OWNS WHAT, BEFORE THREE TEAMS DISCOVER THEY ALL DO

Payment Processing
Payment lifecycle · execution · attempt · capture, refund, void · payment state · acquiring connectivity

Payment Optimization
Objective functions · optimization strategy · economic models · optimization outcomes and their readouts

Decision Platform
Strategy registration · assignment · evaluation · experiment allocation · evidence · versioning · promotion and rollback

Every product team
The decision it wants made, and the strategy that makes it

THE SYMMETRICAL WARNING
We already say an orchestration engine that accumulates business logic becomes the monolith. The same is true of the decision platform — and it is the more likely of the

— The test: if the decision platform starts holding business rules rather than executing registered strategies, it has become the new central authorization engine under a better name.

THE QUESTION	THE ANSWER
Who decides what the routing strategy should be?	Payment Optimization — it owns the objective function
Who writes the strategy artefact?	The product team that owns the decision — usually Payment Processing for routing
Who executes it, assigns it and records the evidence?	Decision Platform, always, with no exceptions
Who owns the transaction state it runs inside?	Payment Processing
Who owns whether it worked?	Payment Optimization, measured against the holdout
Who can change the objective?	Product and Finance, through the exchange rate — not an engineer, and not per deal

THE FAILURE THIS PREVENTS

Without the split, the likeliest outcome is that the decision platform accumulates business logic — first a rule, then an exception, then a population-specific override — until it is the central authorization engine again, with a better name and the same problem.

The test is one question, asked quarterly: ***does the decision platform hold business rules, or does it execute registered strategies?*** The first answer means the boundary has already gone.

The product team charter

> The seven obligations describe what a product owes. This describes who inside the team owes it. A team missing one of these roles will quietly drop the obligation attached to it.

ROLE	OWNS	THE OBLIGATION AT RISK WITHOUT THEM
Product Manager	Strategy, roadmap, the customer problem, the commercial proposition, metrics, prioritisation, lifecycle	Monetization — the product ships without a commercial model
Product Designer / Experience Lead	Client journey, developer experience, portal experience, operational experience	Setup and integration — the API works and nobody can adopt it
Engineering Lead	Implementation, architecture within the boundary, reliability, performance, technical roadmap	Execution — the SLO becomes aspirational
Data / Decision Lead	Metrics, decision features, experimentation, outcome measurement	Reporting and learning — the product improves on opinion
Product Operations	Launch, adoption, product readiness, servicing processes	Servicing — support inherits a product nobody prepared them for

Six questions every product must answer

#	QUESTION	WHAT A WEAK ANSWER LOOKS LIKE
1	What customer problem do we solve?	A description of what the component does
2	What capability do we uniquely own?	A list of things it touches
3	What is our contract?	“Whatever the consumers need”
4	What metrics define success?	Delivery metrics — releases, story points, uptime
5	What commercial value do we create?	“It is foundational”
6	What operational experience do we provide?	“Support raises a ticket with us”

THE TEST

If a team cannot answer all six, it owns a component rather than a product. That is not a criticism — components are necessary — but it should be named accurately, because a component with a product manager attached gets product expectations and component authority, which is the worst combination available.

Solutions and Product

> Solutions packages what exists into offers a segment will buy. Product builds what exists and owns how well it works. Most friction between the two is a missing interface rather than a personality problem.

SOLUTIONS COMPOSES. IT DOES NOT COMMISSION.

SOLUTIONS

Industry and segment offers · packaging · entitlement patterns · commercial narrative · pricing to market

Owns: what we sell, to whom, packaged how, at what price

Capability Catalog
what the platform can verifiably do

Product & Solution Catalog
what is sellable, and in which package

Entitlement patterns
what a client of this shape gets by default

PRODUCT

Builds capabilities · owns the seven obligations · publishes what the platform can verifiably do

Owns: what exists, how well it works, and what it costs us to run

THE TWO RULES THAT KEEP THE INTERFACE CLEAN

Solutions cannot commission a capability per deal
A missing capability is a roadmap input, not a project. Otherwise the roadmap is written by whichever deal is loudest.

Product cannot sell directly
A capability becomes sellable only when it is in the catalog with a package and a price. Otherwise entitlement drifts from contract.

— Most product-versus-solutions friction is a missing interface, not a personality problem. The three registries in the middle are the interface.

	SOLUTIONS OWNS	PRODUCT OWNS
Question answered	What do we sell, to whom, packaged how, at what price	What exists, how well it works, what it costs us to run
Artefacts	Industry solutions, packages, entitlement patterns, pricing to market, the commercial narrative	Capabilities, the seven obligations, published capability entries, the roadmap
Time horizon	This quarter's pipeline and the segment strategy	The multi-quarter capability roadmap
Success measured as	Win rate, attach rate, revenue per client, time to first transaction	Capability adoption, reliability, unit economics, client outcome
Cannot	Commission a capability for a single deal	Sell directly, outside the catalogue

THE ESCALATION PATH THAT HAS TO EXIST

A deal needs something that does not exist. Without a defined path, one of two things happens: the deal is lost, or a product team builds a one-off and the roadmap is quietly written by whichever deal was loudest.

The path: Solutions raises it as a roadmap input with the revenue and segment case attached. Product decides. If the answer is no and the deal is strategically important enough, that is an executive exception with a named owner and a stated cost — not a sprint decision, and not invisible.

Solution composition, in practice

> Three worked examples. In each case Solutions built nothing — it packaged, priced and named what Product already owned.

THREE SOLUTIONS, COMPOSED FROM THE SAME PRODUCTS

Airline	Marketplace	Retail omnichannel
Payment Processing	Payment Processing	Card Present
Credential Management	Platform Accounts	Ecommerce
Instrument Intelligence	KYB and Boarding	Credential Management
Authentication	Money Movement	Network Tokens
Multi-currency	Split Payments	Identity Resolution
Recurring	Reserves	Authentication
Accelerated Funds	Tax	Optimization
Disputes	Disputes	Loyalty

Solutions did not build any of these. It packaged them, priced them and named them.

— If a solution needs something that does not exist, that is a roadmap input with a revenue case attached — never a project Solutions commissions directly.

SOLUTION	THE SEGMENT PROBLEM IT ANSWERS	WHAT MAKES IT A SOLUTION RATHER THAN A BUNDLE
Airline	Multi-currency ticketing, high-value transactions, chargeback exposure, agency and interline complexity	A default policy set, an entitlement pattern and a pricing model tuned to the segment
Marketplace	Onboarding sellers at volume, splitting funds correctly, tax and 1099, and negative balances	Pre-configured split and reserve logic, plus the boarding journey already assembled
Retail omnichannel	One customer across store and online, and returns that work in either direction	Identity resolution configured for the channel mix, and a card-present estate model

WHAT SOLUTIONS IS NOT

It is not a second product-development organisation, and the fastest way to make it one is to let it commission capabilities per deal. Within two quarters the roadmap is written by whichever deal was loudest and the product organisation is a delivery function.

The escalation path exists for the genuine cases: Solutions raises a roadmap input with the revenue and segment case attached, and Product decides. If the answer is no and the deal matters enough, that is a named executive exception with a stated cost — visible, and rare by construction.

Entitle by pattern, restrict by policy

> The default should be that a merchant can do almost everything, and that what they cannot do is expressed as policy rather than as an absent entitlement. Four patterns, not a per-merchant menu.

ENTITLE BY PATTERN · RESTRICT BY POLICY

Standard
direct merchant, single or multi-acquirer, card and common local methods

Enterprise
multi-entity, multi-currency, forward API, advanced optimization and funds

Platform
marketplace, PayFac and ISV — boarding, split, reserves, instant payout

Infrastructure
other PSPs and banks consuming intelligence, decisioning or vault

FOUR STATES A CAPABILITY CAN BE IN, WITHIN A PATTERN

Available	entitled and usable immediately, no setup step
On activation	entitled, needs a setup step — a bank account, a certification, a terminal
Gated	needs underwriting or compliance clearance before it opens
Policy-restricted	open, but narrowed by regulation, scheme rule or client instruction

— Four patterns, not a per-merchant menu. A la carte entitlement makes every boarding a configuration project, and makes the question “what is this client entitled to” unanswerable at scale.

Why a la carte entitlement is the wrong default

A LA CARTE	BY PATTERN
Every boarding is a configuration project	Boarding picks a pattern and moves on
“What is this client entitled to” requires reading their configuration	It is the pattern, plus a short list of exceptions
Selling a capability requires a config change per client	A capability enters a pattern once
Entitlement drifts from the contract, invisibly	Drift is one list, and it is auditable
Support cannot tell whether a failure is a bug or a missing entitlement	The pattern answers it in one lookup

THE REFINEMENT THAT KEEPS “ENTITLED TO EVERYTHING” SAFE

Entitled is not the same as immediately usable, and collapsing the two creates exactly the boarding friction we are trying to remove — in the other direction.

Four states, all within a pattern. **Available** means usable now. **On activation** means entitled but needing a setup step, such as a bank account, a certification or a terminal. **Gated** means needing underwriting or compliance clearance first. **Policy-restricted** means open but narrowed by regulation, scheme rule or the client’s own instruction.

This is what lets a merchant be entitled to the marketplace product without being able to board sub-merchants before underwriting has cleared — without either blocking the sale or creating a bespoke entitlement.

Product metrics and the scorecard

> One scorecard shape for every product, so that a portfolio conversation is possible at all. Five categories, and the fifth is the one that will feel optional.

ONE SCORECARD SHAPE, EVERY PRODUCT, EVERY QUARTER

CUSTOMER	BUSINESS	PAYMENT OUTCOME	PLATFORM	LEARNING
Adoption	TPV	Authorization	Client-impacting SLO	Experiments run
Time to first transaction	Revenue	Conversion	Latency	Experiment velocity
Usage depth	Margin	Cost per transaction	Availability	Hypotheses confirmed
Retention	Attach rate	Fraud and dispute	Change failure rate	Strategies promoted
Experience	Expansion	Recovery	Recovery time	Evaluable history

— The learning column is the one that will feel optional and is not. A product with no experiments is a product improving on opinion, and it will be out-executed by one that is not.

What each product actually measures

PRODUCT	PAYMENT OUTCOME METRICS	BUSINESS METRICS
Payment Processing	Authorization rate, conversion, latency, retry recovery, failed transactions	TPV, revenue, merchant adoption, time to first transaction
Payment Optimization	Incremental authorization against holdout, recovered transactions, cost saving, net merchant benefit	Attach rate, share of gain captured
Credential Management	Network token penetration, updater success, credential recovery, token lifecycle success	Active credentials, attach rate
Instrument Intelligence	Coverage, accuracy, freshness, decision utilisation, incremental payment performance	Internal consumption, external licensing revenue
Authentication & Risk	Frictionless rate, exemption success, fraud rate, dispute rate, net conversion	Attach rate, loss avoided
Money & Funds	Funding speed by segment, settlement accuracy, reconciliation breaks, payout success	Acceleration revenue, balances held, liquidity utilisation
Platform products	Client-impacting SLO, latency, change failure rate, recovery time	Consuming products, forks avoided, internal adoption

THE METRIC THAT KEEPS THE MODEL HONEST

Time to first transaction. It spans setup, integration, entitlement, boarding, compliance and certification — which means no single team can fix it alone and no single team can hide behind another.

It is the closest thing to a single number that tells you whether the product-centric operating model is actually working. If it improves, the obligations are being owned. If it does not, they are being delegated.

Governance, investment, and the role of architecture

> Product-centric governance replaces project-centric governance, which mostly means fewer forums with more authority rather than the reverse.

The product council

DECIDES	DOES NOT DECIDE
Product strategy and direction	Sprint content or delivery sequencing
Investment level and reallocation	Feature prioritisation within an agreed strategy
Cross-product boundary disputes	Anything the product manager can decide alone
Risk acceptance beyond the PM's mandate	Design choices inside a boundary
Architectural exceptions with a cost	Implementation approach

THE FAILURE MODE OF PRODUCT COUNCILS

They become approval gates. Every decision routes through a forum that meets fortnightly, and the organisation has re-created project governance with a new name and a longer cycle time.

The defence is an explicit default: **the product manager decides unless the decision is on the list above.** A council whose agenda is full is a council doing the PM's job, and that is a symptom to act on rather than a sign of engagement.

Investment

FROM	TO
"What does the authorization engine cost to run?"	"What outcome does Payment Processing produce?"
Allocating to systems	Allocating to products against TPV opportunity, revenue, margin and strategic differentiation
Annual project funding	Persistent product funding with quarterly reallocation
Business case per project	Product scorecard reviewed quarterly

What enterprise architecture owns now

FUNCTION	OWNS	DOES NOT OWN
Architecture	Domain boundaries, technical principles, contracts, resilience, scalability, security, data ownership, platform standards	Product strategy, roadmap or the customer outcome
Product	Customer outcome, product strategy, roadmap, commercial proposition	System design inside the boundary
Engineering	Implementation, reliability, technical execution	What gets built and why
Solutions	Market composition, industry proposition, client solution	Capability creation
Operations	Operating execution across products	Product design

The shift for architecture is from designing systems to **creating and protecting the boundaries that let the company move quickly**. That is a smaller remit and a more consequential one: the twelve principles in EA-001 become operating-model rules rather than architecture-only rules, and architecture's job is to hold them when a delivery date makes breaking one attractive.

Business operations

> Operations needs client state, not system state. That distinction decides whether servicing scales with volume or with headcount.

OPERATIONAL QUESTION	WHAT IT NEEDS	WHO OWNS PROVIDING IT
Why is this merchant not live yet	Boarding state across every product's setup steps, with the blocker named	Boarding & Servicing orchestrator, fed by each product
Why did this transaction fail	The decision trace across every product it touched	Decision platform, surfaced by each product
Why is this invoice wrong	Metering to rating to invoice, traceable per line	Monetization & Metering aaS
Why has this merchant not been funded	Settlement position, funding state and any hold, with the reason	Funds Management
What is this client entitled to	Pattern, exceptions and activation state	Entitlement service
What changed for this client, and when	A change history across products, bitemporal	Control plane

THE OPERATIONS TEST FOR ANY NEW PRODUCT

Before a product ships, a non-engineer must be able to answer all six questions above for a client using it, without an escalation. If they cannot, obligations six and seven are not met and the product is not finished — whatever the runtime does.

This is worth enforcing as a launch gate rather than a guideline, because it is always the first thing traded away under a date.

Technology operations

> In a distributed architecture, a wall of green service indicators tells you nothing about whether a merchant can take a payment. The unit of operational observation has to be the client outcome.

IN A DISTRIBUTED ARCHITECTURE, 71 GREEN SERVICES TELL YOU NOTHING

Can this client transact, right now
per-client health, not per-service health

Why did this transaction do that
one decision trace across every product it touched

Is this client set up correctly
entitlement, configuration and activation state in one view

Is the money right
metering to rating to invoice to ledger, reconciled and monitored

What is this incident costing
impact expressed in affected clients and volume, not services

Which product owns this
every alert routes to a product team, never to a shared queue

AND THE TWO OPERATING SURFACES THAT CONSUME IT

Business operations
boarding, servicing, disputes, billing enquiries, client change — needs client state, not system state

Technology operations
availability, latency, error budgets, change and incident — needs system state expressed as client impact

— The unit of operational observation is the client outcome. Every product publishes client-impacting SLOs, and service uptime is an input to those rather than the thing reported.

WHAT TECH OPS NEEDS END TO END	WHY IT IS HARD IN A DISTRIBUTED ESTATE	WHAT PROVIDES IT
Setup visibility	Configuration lives in every product	Control plane, with one view of resolved state per client
Processing visibility	A transaction crosses six or more products	One correlation identifier, one decision trace, one epoch
Monetization visibility	Metering is emitted by products, rated centrally	Metering pipeline health as a first-class SLO
Reporting visibility	Every product reports its own slice	Reconciliation between product views and the ledger
Impact visibility	Service health does not equal client impact	Client-impacting SLOs published per product
Ownership routing	A shared alert queue means nobody owns it	Every alert routes to a product team by construction

THE OPERATIONAL CONTRACT EVERY PRODUCT PUBLISHES

Not uptime. **Client-impacting SLOs**: what proportion of clients could complete the thing this product exists for, within the latency it promises, in a period. Service availability is an input to that number, not the number.

Alongside it, a stated degradation ladder — what this product does when each of its dependencies is unavailable — and a rehearsal cadence for it. An untested degradation path is an outage with extra steps.

Culture, and why it is downstream of ownership

> Five behaviours. Each is stated with the failure it replaces, because a value without a named failure mode is decoration.

BEHAVIOUR	THE FAILURE IT REPLACES	WHAT MAKES IT REAL
Own the whole thing	“That is billing’s problem”	The seven obligations, enforced at launch review
Be obsessed with the client, not the stakeholder	Roadmaps driven by internal escalation rather than client outcome	Client-impacting SLOs and outcome measures, not feature counts
Prove it	Shipping on intuition and reporting the favourable half	The holdout, and readouts that include cost alongside approval
Make the call	Decisions escalated because nobody is sure they own them	Named owners per decision, published; escalation as an exception with a cost
Leave the door open	Retention through lock-in rather than through value	Forward API. The posture has to be real to be credible internally as well as externally

THE HONEST VERSION

The silo problems this organisation has had are not a values failure. They are what happens when accountability is drawn smaller than the client’s experience — a team owns a service, the client experiences a product, and the space between them belongs to nobody.

Section 15 is the fix. This section is what becomes possible once the fix is in, and what to watch for as evidence that it is not. The clearest early signal that ownership is still too small is a product team describing a client problem as belonging to another team.

PART

IV — Getting there

34 From engines to products

35 The six migration stages, and why shadow is the one that matters

36 Moving the merchant estate

37 The competitive narrative

38 What has to happen before what

39 FY2027 strategic outcomes

40 What we commit to in FY2027

41 Strategic risks

42 The compounding advantage

43 What success sounds like

The innovation model

> Innovation as a cadence rather than an initiative. Every product runs the same quarterly loop, and the infrastructure to do it already exists.

STEP	WHAT HAPPENS	WHO PROVIDES THE INFRASTRUCTURE
Discover	Customer problems, competitive gaps, scheme changes, emerging methods, operational pain	Product, from its own scorecard and clients
Hypothesise	A single sentence with a population, a treatment, a metric, an effect size and a direction	Product owns the hypothesis
Experiment	Registered, gated, assigned at the divergence point, guardrailed	Experimentation platform
Measure	Approval, cost, fraud, dispute and net — against the holdout	Data platform and decision evidence
Scale	Promote to a named population, recompile the baseline, publish with an expiry	Decision platform

The product opportunity backlog

Every major product maintains one, containing customer problems, competitive gaps, emerging technology, network and scheme changes, monetisation opportunities and operational improvements. It is distinct from the delivery backlog and it is the input to quarterly planning — the delivery backlog is what we decided, the opportunity backlog is what we have not decided yet.

EXPERIMENT VELOCITY IS A PRODUCT METRIC

Not experiments run — experiments *concluded*, with a promote, hold or retire decision recorded. A product running many experiments and concluding none is generating activity, not learning.

Examples worth running early, because each answers a question we currently assert: does processor A outperform B for this instrument class; does network tokenisation raise approval for this population; does retry at ninety seconds beat immediate retry; does 3DS friction reduce net conversion; does accelerated funding improve retention.

From engines to products

> The estate today is a set of engines with an operational perimeter around them. The transition is not a rewrite and not a big-bang cutover; it is six waves, two of which carry the whole plan.

SIX WAVES · AND THE ONE EVERYONE SKIPS

WAVE 0 — **Evidence on the existing estate**
Propensity and fallback logging on the engines we run today. No migration, no merchant change, and it cannot be backfilled later.

WAVE 1 — **New edge and control plane**
Gateway, identity, entitlement, tenancy, capability catalog, policy, publication. Nothing behind it has moved yet.

WAVE 2 — **Capability migration behind the edge**
Strangler, capability by capability. Existing merchants keep their integration and are served by new components without knowing.

WAVE 3 — **New API surface**
The full resource model. New capabilities land here and only here — stated as policy on day one, not decided later.

WAVE 4 — **Merchant cohort migration**
By segment and by value, not alphabetically. Each cohort needs a reason to re-integrate that is worth the cost to them.

WAVE 5 — **Legacy decommission**
The wave that gets deferred. If the engines stay, we run two platforms forever and the cost case never lands.

— Waves 0 and 3 carry the whole plan. Wave 0 because the evidence cannot be created backwards. Wave 3 because a compatibility layer good enough to remove the reason to migrate is a compatibility layer that becomes permanent.

WAVE	MERCHANT IMPACT	WHAT IS TRUE AT THE END
0 · Evidence on the existing estate	None	Every decision on the current engines is logged with propensity and fallback. The holdout exists. Nothing has migrated.
1 · New edge and control plane	None	Admission, identity, entitlement, tenancy, capability, policy and publication exist. Traffic still reaches the engines.
2 · Capability migration	None — existing integrations unchanged	Capabilities are served by new components behind the same contract. Merchants do not know.
3 · New API surface	Opt-in	The full resource model is live. New capabilities land here only.
4 · Cohort migration	Re-integration required	Merchants move by segment and value, each with a reason worth the cost to them.
5 · Decommission	None visible	The engines are off. This is the wave that gets deferred.

THE TWO THAT CARRY THE PLAN

Wave 0, because counterfactual evidence cannot be created backwards. Every month it waits is a month of traffic that can never train or evaluate anything, and the delay compounds — a learned strategy needs roughly three months of logged history before it can be evaluated at all, so delay postpones the benefit by the delay plus three months.

Wave 3, because of a trap that is easy to walk into. A compatibility layer good enough to keep existing merchants comfortable removes their reason to ever migrate. The only defence is a policy stated on day one: ***new capabilities land on the new API and nowhere else***. Decided later, it will be overruled by the first large client who wants a new capability on the old integration.

The six migration stages, and why shadow is the one that matters

> The waves move capability. The stages move merchants. They run in parallel and are often confused, which is how migration plans end up with an engineering schedule and no merchant proposition.

SIX STAGES · AND THE ONE THAT GENERATES THE BUSINESS CASE

1 Observe
Modern platform sees legacy traffic. No behavioural change, no risk, no merchant involvement.

2 Shadow
Modern decisioning evaluates every transaction; legacy still executes. Outcomes compared, per merchant.

3 Pilot
Selected merchants execute through the new platform, with the shadow comparison as the baseline.

4 Expand
Volume increases progressively within each migrated merchant, guardrailed.

5 Default
New platform is the default path for new and migrated traffic.

6 Retire
Legacy capability decommissioned once thresholds are met. The stage that gets deferred.

WHY STAGE 2 IS THE IMPORTANT ONE

It costs the merchant nothing
No integration, no traffic moved, no risk taken

It produces the number
What approval and cost would have been on the new platform, on their own traff

It is per merchant
The migration case is specific, not a platform-wide average

It de-risks stage 3
A pilot that disagrees with shadow is a bug found before traffic moved

— Shadow mode turns “migration should be outcome-based” from an aspiration into a method. Without it, every migration conversation is a promise; with it, it is a measurement.

Shadow mode in detail

WHAT IT DOES	HOW
Evaluates every transaction on the modern platform	The decision platform resolves the flow and records what it would have decided, without dispatching
Leaves execution with legacy	The legacy engine remains the only path to the network
Records both	Legacy decision and outcome, modern decision and its counterfactual, on the same transaction
Produces a per-merchant comparison	Where the two would have differed, how often, and on which population
Cannot prove the outcome	It shows which decisions would change, never whether they would have been approved

THE HONEST LIMIT, AND HOW TO CLOSE IT

Shadow mode answers “what would we have decided differently”. It does not answer “would that have been better” — that is a counterfactual, and only a live experiment can settle it. The two must be reported separately and never merged, exactly as with replay and backtest in EA-001.

The sequence that closes it: shadow produces the candidate population and the size of the change; the pilot runs a real experiment on that population; the readout becomes the migration case for the next cohort. Shadow does not replace the experiment — it makes the experiment worth running on the right merchants.

Migration tooling

TOOL	WHAT IT REMOVES FROM THE MERCHANT'S RISK
Compatibility APIs	The need to re-integrate in order to be served by the new platform
Migration SDKs and adapters	Hand-written translation between the old and new resource models
Transaction replay	"What would have happened to last month's traffic", answered before cutover
Dual-run reporting	Both platforms reporting the same period, reconciled, during the transition
Outcome comparison	A per-merchant view of approval, cost and latency across the two paths
Certification tooling	Self-service certification instead of a scheduled engagement
Sandbox with deterministic simulators	Testing failure modes without a third party's test environment

Moving the merchant estate

> The uncomfortable fact at the centre of this plan: for the full experience, merchants have to re-integrate. Any migration narrative that avoids saying so is one that fails at the first customer conversation.

PATH	WHAT THE MERCHANT DOES	WHAT THEY GET	WHAT IT IS FOR
Compatibility layer	Nothing	Reliability, and our operational benefits	Moving volume onto the new platform without a client project
Selective adoption	Adds one new endpoint	One new capability — optimization, funds acceleration, Forward API	Creating pull, and proving value before asking for a full migration
Full migration	Re-integrates to the new resource model	Everything, including what is not built yet	The end state

Sequencing the cohorts

COHORT	WHY THEM	THE OFFER THAT JUSTIFIES THE WORK
New logos	No migration cost at all	Everything, from day one
Multi-acquirer enterprises	Highest pain today, clearest gain	One integration replacing several, plus measured optimization
Platforms and PayFacs	Fastest-growing, and funding speed matters most to them	Instant seller payout and boarding they do not have to build
Optimization-sensitive merchants	Approval rate is already a board-level metric for them	A measured lift against a holdout on their own traffic
Long-tail stable merchants	Lowest gain, highest relative cost	Compatibility layer, and no migration ask until decommission

WHAT MAKES THE ASK REASONABLE

Not “our platform is more modern”. No merchant re-integrates for our architecture.

The ask is reasonable when we can say, on their own traffic: here is the approval and cost gap between where you are and where you would be, measured against a holdout, and here is how much faster your money arrives. That requires the evidence layer and at least one live cohort first, which is why the order of the waves matters more than their pace.

It also means the first cohort is partly a proof-generation exercise and should be resourced as one, with the readout treated as a deliverable rather than a by-product.

The competitive narrative

COMPETITOR	WHAT THEY ARE OPTIMISED FOR	WHERE OUR MODEL DIFFERS
Stripe	Developer experience, product velocity and ecosystem breadth, with orchestration and acquiring combined inside their own platform economics	We are explicitly designed to operate across multiple acquiring relationships, and to settle and fund as well as route. We will not match their API surface area, and should stop measuring ourselves that way.
Adyen	A single unified platform with broad global acquiring, optimised for merchants who want one provider end to end	Their model is strongest when they are the acquirer. Ours is designed to add value across an estate we do not wholly acquire, and to let credentials move.
Independent orchestrators	Genuine acquirer neutrality, optimised for routing across providers without taking balance-sheet positions	They are more neutral than we are and we should say so. We add settlement, funding and underwriting to the routing layer, which is a different proposition rather than a better version of theirs.
Incumbent acquirers	Scale, relationships and existing volume, optimised for stability on established processing estates	The same position we are transforming out of. The gap is measurability and speed of change, and it is the gap this programme closes.

THE ONE-SENTENCE POSITION

The PSP that makes your acquiring strategy better — even where we are not the acquirer.

Or, as a brand promise: *we do not need all of your acquiring volume to create value from all of it.*

That sentence is only true if Forward API is real, the holdout exists, and funding acceleration is a priced product with a stated time. All three are in the FY2027 commitments for that reason.

What has to happen before what

The ten commitments are not a flat list. They sit on three mostly independent chains, and one of those chains is seven deep.

THREE CHAINS · AND WHAT EACH ONE GATES



ONLY ONE ITEM ON THIS PAGE CANNOT BE STARTED LATER

Evidence logging. Everything else has a later start date that still works.

— The chains are largely independent, which is the good news: funds does not wait for decisioning, and the control chain does not wait for either. The bad news is that the evidence chain is seven deep and its first link is the one most easily deferred.

CHAIN	LENGTH	CAN START	GATES
Evidence	Seven deep	Immediately — on the existing estate	Optimization, instrument intelligence, learned strategies, and the migration business case
Control	Five deep	Immediately — independent of the evidence chain	Every product, every entitlement pattern, every policy
Funds	Six deep	Immediately — independent of both	Accelerated funds, capital products, and the platform segment proposition

THE GOOD NEWS AND THE BAD NEWS

Good: the three chains are largely independent. Funds does not wait for decisioning. The control chain does not wait for either. Three teams can start in parallel without a coordination layer, which is unusual for a programme this size.

Bad: the evidence chain is seven links long and its first link — logging on the engines we already run — is the one most easily deferred, because it is invisible, unglamorous and touches production systems nobody wants to touch. Deferring it does not delay the last link by the length of the delay; it delays it by the delay plus the three months of history a learned strategy needs before it can be evaluated at all.

FY2027 strategic outcomes

> Commitments describe what we will build. These describe what will be true. The measures are fixed here; the targets are set in planning, with Finance and each product owner.

AREA	OUTCOME MEASURE	WHO SETS THE TARGET
Growth	Share of enterprise TPV orchestrated	Product · Commercial
	Non-acquired TPV orchestrated, in dollars	Product · Commercial
	Product attach rate per enterprise client	Solutions · Commercial
Payment performance	Authorization improvement in bps, measured against the holdout	Product · Analytics
	Merchant payment-cost saving, in dollars	Product · Finance
	Share of decisions that are experimentally measurable	Platform
Funds	Share of eligible TPV available at T+0	Funds Management
	Accelerated funds volume, and the revenue on it	Funds Management · Treasury
Platform	Share of new clients onboarded on the new product model	Product · Boarding
	Median time to first transaction, in days	Solutions · Boarding
Migration	Share of strategic TPV migrated	Migration
	Legacy engines decommissioned	Technology

Commitments, with the scale measure attached

COMMITMENT	THE CAPABILITY	THE SCALE MEASURE
1 · Evidence logging and holdout	Live on the existing estate	Share of eligible TPV covered; control population and its statistical power
2 · Economic objective	Expressed as an exchange rate by pricing model	Share of decision points with an objective bound to them
3 · Control plane and entitlement	Live	Share of traffic governed by a published epoch
4 · Decision platform	Two decision points executing registered strategies	Decisions per second; added latency at p99
5 · Forward API	In production with one enterprise merchant	Credentials under management; TPV presented through it
6 · Multi-acquiring	Live across two acquirers	Orchestrated TPV, split acquired and non-acquired
7 · First optimization readout	Against the holdout, with cost alongside approval	Incremental bps, and dollars at current volume
8 · Funds acceleration priced	A product with a stated funding time	Funding volume; revenue; cost of liquidity consumed
9 · Seven obligations	Published by every front-of-house product	Share of products compliant, audited at launch review
10 · Entitlement patterns	Four patterns live	Median time to first transaction

A CAUTION ON ATTACHING NUMBERS TOO EARLY

Several of these measures describe capabilities that do not exist yet, and a target set before the tooling exists becomes the reason the tooling gets skipped. Commitment 4 is the clearest case: a decisions-per-second target set now will be met by a simpler decision platform than the one we need.

The discipline: set the measure now and the target when the first version is measurable. A measure without a target is honest. A target without a measurable thing is theatre.

What we commit to in FY2027

#	COMMITMENT	WHY THIS ONE	OWNER
1	Propensity and fallback logging live on the existing estate, and a 2% holdout reserved	Cannot be created retroactively. Every other commitment depends on it.	Platform · Product
2	The economic objective expressed as an exchange rate, by pricing model	Without it, “optimization” has no definition and no product	Product · Finance
3	Control plane, capability catalog, policy platform and entitlement patterns live	Everything downstream resolves against them	Architecture · Platform
4	Decision platform executing registered strategies for at least two decision points	Turns the optimization pillar from design into mechanism	Platform
5	Forward API in production with at least one enterprise merchant	Makes the multi-acquiring posture credible rather than positional	Credential Vault
6	Multi-acquirer routing live across two acquirers, measured	Pillar one, demonstrated	Payment Processing
7	First measured optimization readout against the holdout, with cost alongside approval	The sentence the whole narrative rests on	Product · Analytics
8	Funds acceleration as a priced product with a stated funding time by segment	Turns a structural advantage into revenue	Funds Management
9	Every front-of-house product publishing all seven obligations and client-impacting SLOs	The operating model, made observable	Product leadership
10	Four entitlement patterns live, with boarding measured in time-to-first-transaction	Removes the boarding friction that blocks every segment	Solutions · Boarding

IF ONLY ONE THING HAPPENS

Commitment 1. It is the least visible item on this page, it will not appear in any client conversation this year, and it is the only one that gets strictly harder every week it is deferred.

Everything else in this document — the intelligence pillar, the optimization pillar, the proof-based narrative, the migration case, the model portfolio in two years — is built on evidence that either starts accumulating now or does not exist when we need it.

PV-2027 v1.0 · Annual Product Vision · 26 September 2026. Implemented by EA-001 v1.1 (enterprise architecture) and EA-002 v1.0 (product architecture). Related: UPOP PRD and architecture suite, POEM experimentation framework, routing strategy PRD and design, policy platform PRD and design, platform governance and operating model, 30/60/90 delivery backlog.

Strategic risks

> Ten ways this strategy fails. Each has an owner and a mitigation that is already somewhere in this document — which is the point of listing them together.

#	RISK	WHY IT MATTERS	MITIGATION	OWNER
1	**Acquiring cannibalisation**	Incentives fight the orchestration strategy and quietly win	Orchestrated non-acquired TPV as a named success measure; the economics in section on economics	Commercial · Product
2	**Merchant migration friction**	The strategy is built and nobody moves to it	Shadow mode produces a per-merchant case before the ask; selective adoption creates pull	Migration · Product
3	**Platform fork**	Product teams bypass shared services and the architecture fragments	Consumer-planned roadmaps plus a visible, time-boxed exception register	Architecture · Platform
4	**Decision platform becomes the monolith**	Recreates the engine problem under a new name	Explicit ownership split; the quarterly test of whether it holds rules or executes strategies	Architecture
5	**Holdout erosion**	Optimization evidence becomes uninterpretable	Immutable creation date; resizing requires named approval, not a quarterly trade	Product · Analytics
6	**Insufficient volume for intelligence**	The behaviour layer never matures and the moat does not form	Sequence high-volume cohorts first; infrastructure segment widens the observation base	Product
7	**Funding economics**	Acceleration destroys margin at scale	Price by liquidity consumed and risk taken, not as a flat product feature	Funds · Treasury
8	**Credit exposure by accident**	A payments team originates lending without that being decided	Capital split out as a separate product under credit risk governance	Risk · Funds
9	**Legacy never retires**	Dual cost persists and the investment case never lands	Explicit decommission thresholds per capability, set at wave 2, not at wave 5	Technology
10	**Solutions bypasses Product**	The roadmap becomes deal-driven	Formal escalation path; commissioning requires a named executive exception with a stated cost	Product · Solutions

THE TWO MOST LIKELY, AND THEY ARE NOT THE TECHNICAL ONES

Risk 1 and risk 10. Both are incentive failures rather than execution failures, both are invisible in a delivery report, and both are resolved by someone senior holding a position when it is expensive to hold.

A programme review that tracks the other eight and not these two will report green until the strategy has already been hollowed out.

The compounding advantage

> Taken individually, every pillar in this document is reachable by a competitor with funding. The combination is not, and that distinction should be stated plainly rather than implied.

PILLAR	ON ITS OWN	WHAT THE COMBINATION ADDS
Bank structure	Banks have it; most do not productise it	Balance sheet joined to payment visibility and underwriting
Multi-acquiring	Orchestrators have it	Joined to acquiring, so we settle as well as route
Credential portability	Technically straightforward	Credible only because retention does not depend on lock-in
Cross-acquirer data	Requires being the orchestration layer	Joined to our own acquiring outcomes as a comparison set
Instrument intelligence	Partly purchasable	The behaviour band is a function of volume, time and the acquirer dimension
Decision platform	Buildable in eighteen to thirty months	Joined to evidence nobody else has been accumulating
Experimentation	Buildable	Joined to a reserved holdout that cannot be created retroactively
Funds acceleration	Banks have the capability	Joined to visibility of the receivable at the moment it is created

> Every transaction creates payment revenue, data, evidence, an optimisation and a funding opportunity — at the same time, from the same event.

WHY THIS IS THE STRATEGY RATHER THAN A SUMMARY

A competitor can match any single row. Matching the combination requires being a bank, holding the orchestration layer, having accumulated the evidence, and having reserved a holdout before the accumulation started — in that order, and the order cannot be compressed.

That is a genuinely defensible position, and it is also fragile in one specific way: it depends on the first link being started now. A compounding advantage that starts a year late does not arrive a year late. It arrives a year late plus the time the compounding would have produced.

What success sounds like

> Three years out, six people should be able to say six things. If any one of them cannot, the corresponding part of this document did not land.

WHO	WHAT THEY SHOULD BE ABLE TO SAY	WHICH PART OF THIS DOCUMENT IT TESTS
A merchant	“I integrate once and I do not need to care which acquirer, processor, token service or rail handles the transaction.”	The forward API model and multi-acquiring
A product manager	“I can launch a new strategy without building a decision engine.”	Decision and experimentation embedded in every product
A solutions leader	“I can compose a new industry solution without waiting for engineering to build a platform.”	Solutions composes, Product builds
An operations leader	“I can trace any merchant issue across setup, processing, settlement, monetisation and reporting from one place.”	Business and technology operations
An engineer	“I own my product end to end without owning every shared capability.”	The seven obligations, and platform products as products
An executive	“Our PSP gets better every time it processes a payment.”	The intelligence loop, and the holdout that makes it provable

THE FOUR-LAYER DOCUMENT MAP

This document is layer one, and it should stay the layer that answers *where are we going and why will customers choose us*. The mechanisms belong below it, and keeping them below it is what stops UPOP, the decision platform or a microservice pattern from accidentally becoming the strategy.

1. Annual product vision — this document. **2. Product architecture** — EA-002, what we build to deliver it. **3. Product operating model** — Part III here, which should be lifted into its own document once it stabilises. **4. Enterprise architecture** — EA-001, the technical foundation.